

TRABAJO PRÁCTICO 5: Optimización y Procesamiento de Datos

Objetivo:

Analizar y mejorar el rendimiento de bases de datos NoSQL mediante la creación de índices, el uso de herramientas de diagnóstico y la implementación de pipelines de agregación complejos.

Formato de entrega:

- Documento PDF con las respuestas teóricas.
- Capturas de pantalla de los comandos ejecutados en mongosh.
- Código de los comandos en formato texto.

ACTIVIDAD 1: Optimización mediante Índices

1) Una empresa de logística gestiona miles de envíos. Las consultas más frecuentes buscan paquetes por su **numero_seguimiento** (único) y otras filtran por estado del envío y **ciudad_destino** simultáneamente.

- a) Identificar qué campo requiere un Índice Simple y proponer el comando para crearlo.
- b) Identificar qué campos requieren un Índice Compuesto para optimizar las búsquedas por estado y ciudad. Proponer el comando.
- c) Explicar qué sucede internamente en MongoDB cuando se realiza una búsqueda en una colección de gran volumen que no posee índices.

2) Implementación práctica:

- a) Crear la base de datos **logistica_db** y la colección **envios**.
- b) Insertar al menos 10 documentos con los campos: **numero_seguimiento**, **estado**, **ciudad_destino** y **peso**.

ACTIVIDAD 2: Diagnóstico y Evaluación de Rendimiento

1) El equipo de desarrollo nota que una consulta específica está demorando demasiado tiempo en ejecutarse.

- a) Mencionar qué comando de MongoDB permite visualizar si una consulta está realizando un escaneo completo de la colección (**COLLSCAN**) o utilizando un índice (**IXSCAN**).
- b) ¿qué significan los campos **executionTimeMillis** y **totalDocsExamined**?

- c) Según la tabla de problemas detectados, si se observa un **uso alto de memoria** debido a documentos muy grandes, ¿cuál es la solución recomendada?

2) Implementación práctica:

- a) Ejecutar una consulta sobre la colección envíos utilizando `explain("executionStats")` para buscar un paquete por su `numero_seguimiento`.
- b) Analizar el resultado: Identificar el stage y verificar si se utilizó el índice creado en la Actividad 1.
- c) Ejecutar el comando `stats()` sobre la colección y reportar el tamaño total de los índices (`totalIndexSize`).

ACTIVIDAD 3: Procesamiento Avanzado con Agregaciones

- 1) Se requiere generar reportes financieros y operativos de los envíos realizados.
- a) ¿Cuál es la función del Pipeline de Agregación y qué significa que funcione como una "tubería"?
- b) Describir brevemente para qué sirven las etapas `$match`, `$group` y `$project`.
- c) ¿Qué operador de agregación utilizarías para unir la colección envíos con una colección de clientes?

2) Implementación práctica:

- a) Realizar una agregación que filtre (`$match`) los envíos con estado "Entregado" y los agrupe (`$group`) por `ciudad_destino` para contar cuántos se entregaron en cada ciudad.
- b) Utilizar una función matemática para calcular el peso total de todos los envíos en la colección.
- c) Crear un pipeline que utilice `$project` para mostrar el `numero_seguimiento` en mayúsculas (usando `$toUpper`) y el estado.
- d) Implementar una etapa de `$addFields` o `$project` que use un condicional `$cond` para crear un campo nuevo llamado "tipo_carga": si el peso es mayor a 20, "Pesada", de lo contrario "Ligera".